argot — C Argument Parsing Library

Software Design Document · draft · Oct 11, 2026 · seesee010

This document is AI-generated. It was written by Claude Opus 5.5 (thinking level "High") in collaboration with seesee010.

1. Introduction

argot is a C99 library that turns `argv` into typed, validated values with one declaration table and one function call — while every stage (tokenizing, conversion, validation, help, errors, memory) can be replaced or extended without forking the library.

1.1 Purpose

This document defines the architecture, public API, parsing semantics and quality requirements for argot 0.1.0. It is the reference for implementation, code review and the test suite.

1.2 Scope

In scope: POSIX/GNU-style command lines, subcommands, typed values, validation, help/usage/version output, shell completions, error reporting, environment-variable fallback.

Out of scope: config file formats (reachable through the value-source hook), interactive prompts, Windows `/option` syntax, and Windows as a platform (planned for 0.2.0).

1.3 Name

Argot means the specialised vocabulary of a group — here, the vocabulary of a tool. Public prefix: `argot_` for functions and types, `ARGOT_` for macros and constants. Working name.

1.4 Glossary

Term	Meaning
Argument	One element of `argv`
Option	A named argument starting with `-` or `--`

Term	Meaning
Short option	One-character option: `-v`
Long option	Multi-character option: `--verbose`
Flag	Option that takes no value
Value	Data attached to an option (`-o file`, `--out=file`)
Positional	Argument that is not an option and not consumed as a value
Subcommand	Positional that selects a nested command spec (`git commit`)
Spec	Tree of items describing the whole command line
Compiled spec	Spec after checking and index building, ready for lookup
Type	Vtable that converts text into typed storage
Hook	User callback that replaces a default behaviour
Arena	Byte buffer the library carves its working memory from
Item	One entry in a command's declaration: option, argument, subcommand, alias, rule, example

2. Goals, non-goals and design principles

The core promise: a working CLI in under ten lines with zero setup, and no ceiling — the simple API is built on the advanced one, so nothing is a dead end.

2.1 Goals

ID	Goal
G1	**Trivial start.** Three typed options in ⩽ 10 lines; nested subcommands without a single struct or variable. No init, no free, no allocation.

ID	Goal
G2	**Progressive depth.** Every default is replaceable through a documented hook. The easy layer only calls public API of the deeper layers — no hidden second-class path.
G3	**Speed.** One linear pass over `argv`, O(1) short-option lookup, no heap on the default path.
G4	**Safety.** No undefined behaviour on any input. Never writes outside caller memory. `argv` is read-only. Fuzzed continuously.
G5	**Portability.** C99 core, freestanding-capable, compiles as C++, no global state, no locale dependency.
G6	**Good end-user UX.** GNU-compatible syntax, readable help, precise errors, “did you mean” suggestions, shell completions.
G7	**Easy to ship.** Single-header mode and classic library mode from the same sources.

2.2 Non-goals

- Not a config system. Config files plug in through the value-source hook.
- Not a translation framework. All messages are overridable, but no catalogue ships.
- Not a binary drop-in for `getopt`. A thin compatibility shim may come later.
- No runtime reflection beyond what the spec itself carries.

2.3 Design principles

1. **Declarative first.** You describe the command line; you do not write the loop.
2. **Specs are plain data.** `const` structs, placeable in `.rodata`/ROM, generatable by tools.
3. **Layered.** Level 0 `ARGOT_RUN` → Level 1 item arrays + `argot_parse()` → Level 2 tokenizer. Each layer uses only the public API of the layer below.
4. **Pay for what you use.** No `malloc`, no `stdio`, no float code unless a feature you use needs it; compile-time switches strip the rest.

5. **Explicit ownership.** Strings are borrowed from `argv`. The library frees only what it allocated itself.
6. **Stop on error by default.** On a user error argot prints the message and exits with its code. Defining `ARGOT_EXIT_ON_ERROR` as 0 makes every entry point return the status instead; `abort()` is never called.
7. **Fail loud on programmer errors.** A broken spec (duplicate keys, unparsable defaults or bounds) is caught when the spec is compiled, with an exact diagnostic.
8. **Least surprise.** Follow POSIX Utility Syntax Guidelines and GNU long-option conventions by default.

3. Requirements

28 functional and 11 non-functional requirements; *Must* items define 0.1.0, *Should* items target 0.1.0, *Could* items may slip to a later 0.x release.

3.1 Functional

ID	Requirement	Priority
FR-01	Short options `-v`, bundling `-abc`	Must
FR-02	Short option values attached `-ofile` or separate `-o file`	Must
FR-03	Long options `--name`, `--name=value`, `--name value`	Must
FR-04	`--` ends option parsing	Must
FR-05	A lone `-` is a positional (stdin convention)	Must
FR-06	Automatic `--no-name` for negatable booleans	Should
FR-07	Counters: `-vvv` → 3	Must
FR-08	Repeatable options collect lists (`-I a -I b`), optional split on a delimiter (`--tags=a,b`)	Must
FR-09	Typed values: bool, signed/unsigned 8–64 bit, double, string, enum, size (`4k`, `16M`), duration (`1m30s`)	Must

ID	Requirement	Priority
FR-10	Optional values `--color[=WHEN]`	Should
FR-11	Positionals: required, optional, variadic with min/max count	Must
FR-12	Subcommands with arbitrary nesting and aliases	Must
FR-13	Defaults and env-var fallback; precedence CLI > sources > env > default	Must
FR-14	Constraints: required, mutually exclusive, requires, any-of, one-of, range, custom	Must
FR-15	Automatic `-h/--help` and `--version`	Must
FR-16	Help generation: wrapping, groups, hidden options, defaults, env names, choices, examples	Must
FR-17	“Did you mean …?” suggestions for options and subcommands	Should
FR-18	Unique-prefix abbreviation of long options (opt-in)	Could
FR-19	Response files `@file` (opt-in)	Could
FR-20	Completion scripts for bash, zsh, fish	Should
FR-21	GNU interleaving or POSIX-strict ordering, selectable	Must
FR-22	Flags that run a function: once and exit (ARGOT_DO) or on every occurrence (ARGOT_ON)	Must
FR-23	User-defined value types	Must
FR-24	Deprecated aliases that still work and print a warning	Could
FR-25	Query whether an option was set and from which source	Must
FR-26	Read values by name in handlers, without declaring variables	Must

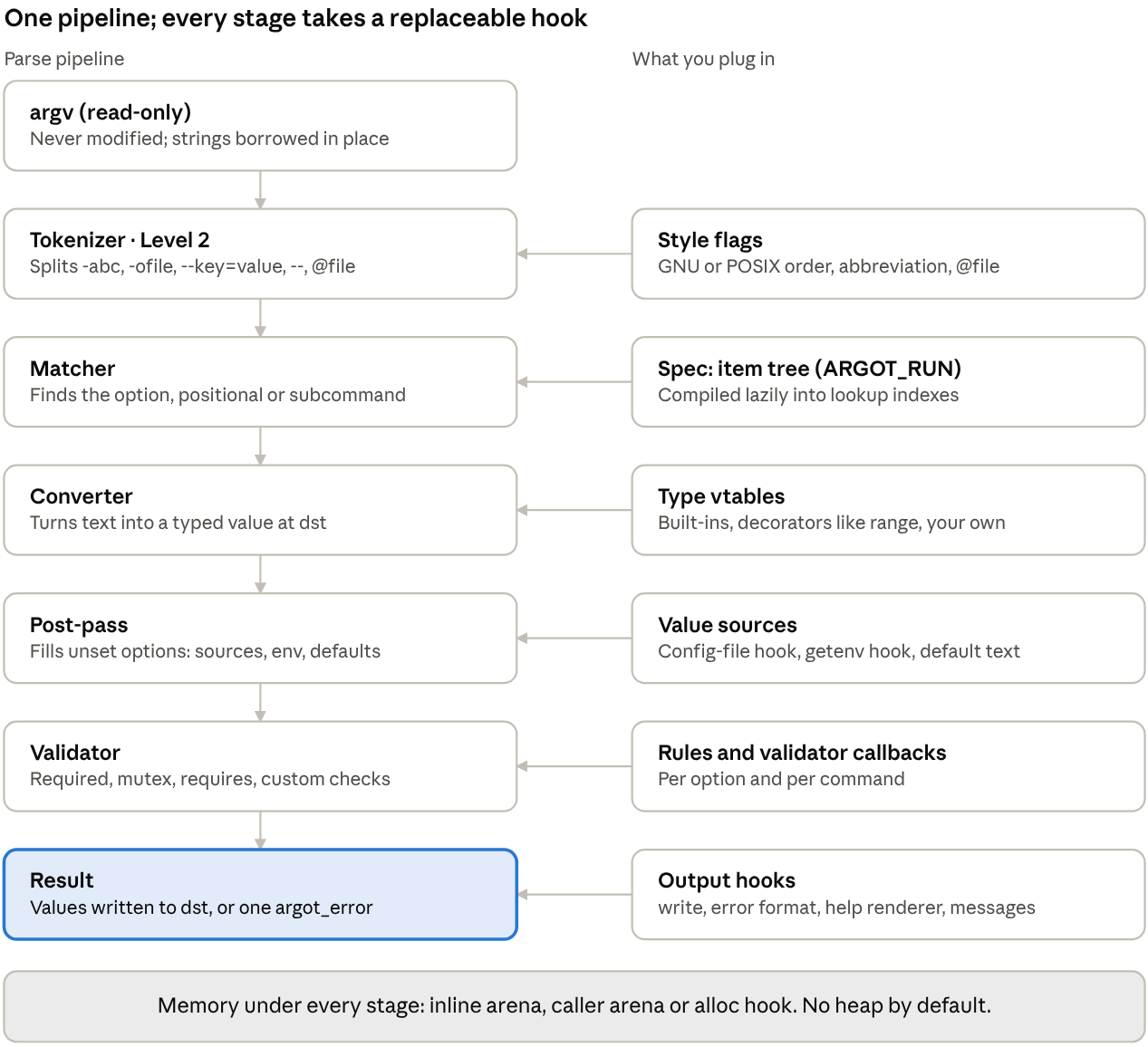
ID	Requirement	Priority
FR-27	Nested declaration: everything that belongs to a command is written inside it	Must
FR-28	Split one command tree across source files	Should

3.2 Non-functional

ID	Requirement
NFR-01	C99 core; C11/C23 extras behind feature checks; compiles cleanly as C++11 and later
NFR-02	No heap use on the core parse path; allocator hook is optional
NFR-03	No mutable global or static state; reentrant; distinct parsers are thread-safe
NFR-04	`argv` is never modified or permuted
NFR-05	Parse time linear in total argument bytes; short lookup O(1), long lookup O(log m)
NFR-06	Freestanding build without `stdio.h` / `stdlib.h` (`ARGOT_FREESTANDING`)
NFR-07	Core `.text` size budget set and tracked per release from M1 on
NFR-08	Zero warnings with `-Wall -Wextra -Wpedantic -Wconversion -Wshadow` (GCC, Clang) and `/W4` (MSVC, from 0.2.0)
NFR-09	Fuzzed with libFuzzer and AFL++; clean under ASan, UBSan, MSan
NFR-10	Test coverage targets: ≥ 95 % lines, ≥ 90 % branches
NFR-11	License fixed by the project owner (see repository)

4. Architecture overview

argot is one pipeline — tokenize, match, convert, fill, validate — fed by a `const` spec, with a hook at every stage and a single memory policy underneath.



argot architecture · 7 pipeline stages, 6 hook points

Read top to bottom for what happens to `argv`; read right to left for where you can change it. ARGOT_RUN and hand-written item arrays produce the same spec; Level 2 users call the tokenizer directly and skip the rest.

Module	File	Depends on
Tokenizer (Level 2)	`tokenizer.c`	—
Spec compiler + matcher	`parse.c`	tokenizer, types
Types and decorators	`types.c`	—
Rules and validation	`rules.c`	parse
Help and usage	`help.c`	parse, types
Completion	`complete.c`	help model
Errors and messages	`error.c`	—
Suggestions	`suggest.c`	—
Windows argv helper	`win.c`	—

Only the tokenizer, parser, types and errors are mandatory; the strip macros in §16 remove the rest.

5. Data model

One struct describes everything the user declares: `argot_item`. An option, a positional, a subcommand, an alias, a rule and a help example are all items, and a command is an item that holds a list of items. Besides it there are only `argot_type` (the value extension point) and `argot_parser` (the only mutable object).

5.1 Item

```
typedef enum argot_item_kind {
    ARGOT_I_END = 0,   /* terminates every item list               */
    ARGOT_I_FLAG,      /* bool, no value                           */
    ARGOT_I_COUNT,     /* +1 per occurrence (-vvv)                 */
    ARGOT_I_VALUE,     /* typed value; .many = true collects a list */
    ARGOT_I_DO,        /* flag that runs a function, then exits    */
    ARGOT_I_ON,        /* runs a function on every occurrence      */
    ARGOT_I_ARG,       /* positional                               */
    ARGOT_I_CMD,       /* subcommand: holds child items            */
    ARGOT_I_ALIAS,     /* extra name for the enclosing command     */
    ARGOT_I_GROUP,     /* starts a help section                    */
```

```
    ARGOT_I_RULE,      /* mutex, requires, one-of, custom ...       */
    ARGOT_I_EXAMPLE,   /* example line for --help                   */
    ARGOT_I_USE,       /* splices in a tree defined elsewhere       */
    ARGOT_I_MAIN       /* handler of the enclosing command          */
} argot_item_kind;

typedef struct argot_item {
    /* set by the macro's fixed arguments */
    argot_item_kind          kind;
    char                     short_name;   /* 0 = none
*/
    const char              *name;         /* long / positional / command
name; NULL = none */
    const char              *help;
    const argot_type        *type;         /* NULL = derived from the macro
(bool, string) */
    union {
        argot_run_fn   run;                /* CMD:  int fn(argot *a)
*/
        argot_do_fn    on_do;              /* DO:   int fn(argot *a)
*/
        argot_on_fn    on_each;            /* ON:   int fn(argot *a, const
char *value)   */
        argot_rule_fn  rule;               /* RULE: custom check
*/
    } fn;
    const struct argot_item *items;        /* CMD children / USE target, END-
terminated   */
    const char *const       *names;        /* RULE members, NULL-terminated
*/
    argot_rule_kind          rule_kind;

    /* optional fields — zero / NULL always means "off" */
    const char *def, *min, *max;           /* text, parsed by .type like user
input      */
    const char *env;                       /* env var fallback
*/
    const char *value;                     /* metavar shown in help: "DIR"
*/
    const char *id;                        /* lookup key override
*/
    const char *alias;                     /* extra long name for an option
*/
```

```
    const char *deprecated;                /* "use --output": still works,
warns          */
    size_t      max_count;                 /* ARG lists: 0 = unlimited
*/
    char        sep;                       /* split "a,b" on this char; 0 =
no split      */
    unsigned    global : 1, hidden : 1, required : 1, many : 1, once : 1,
                negatable : 1, optional_value : 1, check_on_do : 1;
    argot_check_fn check;                  /* per-item validator
*/
    argot_to       to;                     /* optional binding: { void *ptr;
int type_id } */
    void          *udata;

    /* set only by the macro name */
    size_t      min_count;                 /* ARG: 1 for ARGOT_ARG(S), 0 for
..._OPT      */
} argot_item;
```

The struct is deliberately wide: one shape for everything keeps the macros trivial and lets items nest freely. It costs some `.rodata` per item, not RAM — specs stay `static const`.

5.2 The “omitted means off” rule

Every field the user leaves out is zero, and zero always means off. Where 0 would be a meaningful value, the field is either **text** or **encoded in the macro name**:

Field	Omitted means	Why it is not a plain number
`short_name` (`0` / `ARGOT_NO_SHORT`)	no `-x`	— (0 is never a valid option char)
`name` (`NULL` / `ARGOT_NO_LONG`)	no `--xyz`	—
`.def`, `.min`, `.max`	no default / no bound	`"0"` is a real bound, so these are text parsed by `.type` — also works for `"16G"`, `"1m30s"`
`min_count`	—	lives in the macro name: `ARGOT_ARG` vs `ARGOT_ARG_OPT`

Field	Omitted means	Why it is not a plain number
`.max_count`	unlimited	a list with max 0 is meaningless, so 0 can mean “no limit”
CMD function (`NULL`)	grouping command; a subcommand becomes mandatory	—

An option with neither a short nor a long name, or a CMD with neither a function nor children, is a spec error reported at startup.

5.3 Lookup keys

Getters find values by key, searching the current command first and then its ancestors (that is how `.global` options are reached):

1. `.id` if set,
2. else the long name (`"dry-run"`),
3. else `"-"` + the short name (`"-n"`),
4. positionals and commands by their name.

Duplicate keys within one command are a spec error.

5.4 Parser and handle

```
typedef struct argot_parser {
    struct argot_parser_priv_ _p;                  /* opaque by convention
*/
    unsigned char arena_inline[ARGOT_INLINE_ARENA]; /* default 1024 bytes
*/
} argot_parser;

typedef struct argot_parser argot;   /* what handlers receive: int fn(argot
*a) */
```

The parser size is public so it can live on the stack; its fields are private and may change between minor versions. Private state per parse: selected command path, per-item state (seen bit, 2-bit source, occurrence count), stored values for unbound items, lazily built indexes, the current error and the arena cursor.

6. Public API

The whole program is one nested declaration: `return ARGOT_RUN(...)` with items inside, commands as items that contain items, and handlers that read values by name — no structs to wire, no variables to declare. Two lower levels stay available: hand-written item arrays with `argot_parse()` (Level 1) and the pull tokenizer (Level 2).

6.1 Level 0 — `ARGOT_RUN`

```
#include "argot.h"

static int install(argot *a)
{
    for (size_t i = 0; i < argot_count(a, "name"); i++)
        printf("install %s%s\n", argot_str_at(a, "name", i),
               argot_flag(a, "force") ? " (forced)" : "");
    return 0;                                   /* becomes the exit code */
}

static int remove_pkgs(argot *a)  { /* ... */ return 0; }
static int repo_list(argot *a)    { /* ... */ return 0; }
static int list_formats(argot *a) { puts("tar zip zst"); return 0; }

static int repo_add(argot *a)
{
    if (argot_flag(a, "verbose"))               /* global flag, inherited */
        puts("adding repo ...");
    printf("%s -> %s\n", argot_str(a, "name"), argot_str(a, "url"));
    return 0;
}

int main(int argc, char **argv)
{
    return ARGOT_RUN(argc, argv, "pkg", "A tiny package manager.",
        ARGOT_FLAG('v', "verbose", "Print more", .global = true),
        ARGOT_STR ( 0 , "root", "Install prefix", .def = "/", .env =
"PKG_ROOT"),
        ARGOT_DO  ( 0 , "list-formats", list_formats, "Show formats and
exit"),

        ARGOT_CMD("install", install, "Install packages",
            ARGOT_ALIAS("i"),
```

```
            ARGOT_FLAG('f', "force", "Reinstall even if present"),
            ARGOT_ARGS("name", "Packages to install")),

        ARGOT_CMD("remove", remove_pkgs, "Remove packages",
            ARGOT_ALIAS("rm"),
            ARGOT_ARGS("name", "Packages to remove")),

        ARGOT_CMD("repo", NULL, "Manage repositories",
            ARGOT_CMD("add", repo_add, "Add a repository",
                ARGOT_ARG("name", "Repository name"),
                ARGOT_ARG("url",  "Repository URL")),
            ARGOT_CMD("list", repo_list, "List repositories")));
}
```

`ARGOT_RUN` parses, prints help/version/errors, runs the selected handler and returns the exit code: 0 for `--help` / `--version`, 2 for usage errors, otherwise the handler's return value. `ARGOT_RUN_EX(&cfg, argc, argv, ...)` takes a config (§6.7). The smallest program is the same thing without commands:

```
int main(int argc, char **argv)
{
    return ARGOT_RUN(argc, argv, "shrink", "Makes images smaller.",
        ARGOT_FLAG('v', "verbose", "Print what happens"),
        ARGOT_INT ('q', "quality", "Quality", .def = "80", .min = "1", .max =
"100"),
        ARGOT_ARGS("image", "Images to shrink"),
        ARGOT_MAIN(shrink_main));        /* handler of the root command */
}
```

6.2 Item catalogue

Macro	Fixed arguments	Declares
`ARGOT_FLAG`	short, long, help	bool flag
`ARGOT_COUNT`	short, long, help	counter (`-vvv` → 3)
`ARGOT_STR`, `ARGOT_INT`, `ARGOT_UINT`, `ARGOT_DOUBLE`, `ARGOT_SIZE`, `ARGOT_DURATION`	short, long, help	typed value

Macro	Fixed arguments	Declares
`ARGOT_ENUM`	short, long, help, `ARGOT_CHOICES("png", "jpg", ...)`	one of a fixed set
`ARGOT_CUSTOM`	short, long, type, help	value of a user type (§8)
`ARGOT_DO`	short, long, function, help	flag that runs a function and exits (§6.5)
`ARGOT_ON`	short, long, function, help	function on every occurrence (§6.5)
`ARGOT_ARG` / `ARGOT_ARG_OPT`	name, help	exactly one / zero or one positional
`ARGOT_ARGS` / `ARGOT_ARGS_OPT`	name, help	one or more / zero or more positionals
`ARGOT_CMD`	name, function or `NULL`, help, child items	subcommand
`ARGOT_MAIN`	function	handler of the enclosing (root) command
`ARGOT_ALIAS`	name	extra name for the enclosing command
`ARGOT_GROUP`	title	help section for the items that follow
`ARGOT_MUTEX`, `ARGOT_ONE_OF`, `ARGOT_ANY_OF`, `ARGOT_REQUIRES`, `ARGOT_CONFLICTS`, `ARGOT_RULE_FN`	keys / function	command rules (§10)
`ARGOT_EXAMPLE`	command line, explanation	example in `--help`
`ARGOT_USE`	tree name	splices in a tree from another file (§6.6)

Every macro accepts optional trailing fields (§6.3). A repeatable option is any value item with `.many = true`: `ARGOT_STR('I', "include", "Search path", .many = true)`.

6.3 Optional fields

All optional, all off when omitted (§5.2).

Field	Applies to	Effect
`.def = "4"`	values, args	default, parsed by the item's type
`.min = "1"`, `.max = "64"`	numeric, size, duration	inclusive bounds, as text
`.env = "APP_JOBS"`	values, flags	env-var fallback
`.required = true`	options	must be given (CLI or env)
`.global = true`	options	valid in every subcommand, reachable from every handler
`.many = true`, `.sep = ','`	values	collect a list; optionally split `a,b,c`
`.once = true`	values	repeating it is an error
`.negatable = true`	flags	adds `--no-<name>`
`.optional_value = true`	values	`--color` or `--color=always`
`.value = "DIR"`	values, `ON`	metavar in help
`.hidden = true`	any	parsed but not listed in help
`.alias = "colour"`	options	extra long name
`.deprecated = "use --out"`	options	works, prints a warning
`.id = "dry"`	any	lookup key override (§5.3)
`.type = &argot_t_int`	args	typed positional (default: string)
`.max_count = 3`	`ARGS`	upper bound on count
`.check = fn`	any	per-item validator

Field	Applies to	Effect
`.to = ARGOT_TO(&var)`	values, flags	also write the value into a variable

6.4 Reading values

Getter	Returns
`argot_flag(a, key)`	`bool`
`argot_int(a, key)`, `argot_uint(a, key)`, `argot_double(a, key)`	`int64_t`, `uint64_t`, `double`
`argot_str(a, key)`	`const char *`, `NULL` if unset and no default
`argot_enum(a, key)`	index into the choices
`argot_count(a, key)`	number of values (`.many`, `ARGS`) or occurrences (`COUNT`)
`argot_str_at(a, key, i)`, `argot_int_at(...)` …	element `i` of a list
`argot_get(a, key, &out)`	value of a custom type, copied into `out`
`argot_is_set(a, key)`	given via CLI, source or env (not via default)
`argot_source(a, key)`	`ARGOT_SRC_CLI`, `_SOURCE`, `_ENV`, `_DEFAULT`, `_NONE`
`argot_path(a)`	selected command path, e.g. `"pkg repo add"`
`argot_udata(a)`	the `udata` pointer from the config

Keys follow §5.3. An unknown key or a getter that does not match the item's type (`argot_int` on a string option) trips `ARGOT_ASSERT` with the key and command in debug builds and returns a zero value in release builds. Strings are borrowed from `argv` or the arena and stay valid while the handler runs.

Optional binding. Whoever prefers variables adds `.to`:

```
static long jobs;
ARGOT_INT('j', "jobs", "Parallel jobs", .def = "4", .to = ARGOT_TO(&jobs)),
```

`ARGOT_TO` uses `_Generic` (C11) to record the pointee type; C99 code uses `ARGOT_TO_LONG(&jobs)` and friends. A mismatch between the variable and the item's type is a spec error at startup.

6.5 Flags that run functions

	`ARGOT_DO`	`ARGOT_ON`
Signature	`int fn(argot *a)`	`int fn(argot *a, const char *value)`
Runs	once, after parsing, instead of the command handler	at every occurrence, during parsing, in `argv` order
Afterwards	program ends; return value = exit code	parsing continues; non-zero aborts with `ARGOT_E_ACTION`
Values available	all of them	those given earlier on the command line
Required/rule checks	skipped, like `--help` (`.check_on_do = true` keeps them)	run normally at the end
Typical use	`--list-formats`, `--dump-config`, `--licenses`	`-I dir`, `--load plugin`, `-D KEY=VAL`

```
static int add_include(argot *a, const char *dir)
{
    if (!is_dir(dir)) return argot_fail(a, "not a directory");
    return add_search_path(dir);
}

ARGOT_ON('I', "include", add_include, "Add a search path", .value = "DIR"),
ARGOT_ON('q', "quiet",   go_quiet,    "Silence output"),        /* no .value:
dir == NULL */
ARGOT_DO( 0 , "list-formats", list_formats, "Show formats and exit"),
```

If several `DO` flags are given, the first one on the command line runs.

6.6 Splitting a tree across files

```
/* repo.c */
ARGOT_DEFINE(repo_cmd,
    ARGOT_CMD("repo", NULL, "Manage repositories",
        ARGOT_CMD("add",  repo_add,  "Add a repository",
            ARGOT_ARG("name", "Repository name"),
            ARGOT_ARG("url",  "Repository URL")),
        ARGOT_CMD("list", repo_list, "List repositories")));

/* main.c */
ARGOT_EXTERN(repo_cmd);

int main(int argc, char **argv)
{
    return ARGOT_RUN(argc, argv, "pkg", "A tiny package manager.",
        ARGOT_FLAG('v', "verbose", "Print more", .global = true),
        ARGOT_USE(repo_cmd));
}
```

6.7 Level 1 — explicit control

The same items work as a `static const` array, for people who want to drive the steps themselves, keep specs in ROM or generate them with a tool:

```
static const argot_item app[] = {
    ARGOT_APP("pkg", "A tiny package manager.",
        ARGOT_FLAG('v', "verbose", "Print more", .global = true),
        ARGOT_CMD("install", install, "Install packages",
            ARGOT_ARGS("name", "Packages to install")))
};

int main(int argc, char **argv)
{
    argot_config cfg = ARGOT_CONFIG_INIT;
    cfg.version_text = "pkg 1.0.0";
    cfg.udata        = &app_state;

    argot_parser p;
    argot_init(&p, app, &cfg);
    argot_status st = argot_parse(&p, argc, argv);
    if (st != ARGOT_OK)
```

```
        return argot_report(&p);        /* only reached with
ARGOT_EXIT_ON_ERROR 0 */
    return argot_run(&p);               /* calls the selected handler */
}
```

The config carries every hook from §8.5:

```
typedef struct argot_config {
    uint32_t            version;        /* set by ARGOT_CONFIG_INIT */
    uint32_t            style;          /* ARGOT_STYLE_GNU |
ARGOT_STYLE_POSIX | ARGOT_ABBREV |
                                           ARGOT_RESPONSE_FILES |
ARGOT_NO_AUTO_HELP | ... */
    const char         *version_text;   /* enables -V/--version */
    void               *udata;          /* returned by argot_udata() */
    argot_write_fn      write_out, write_err;  void *io_ud;
    argot_getenv_fn     getenv;         void *env_ud;
    argot_alloc_fn      alloc;          void *alloc_ud;
    argot_source_fn     sources[ARGOT_MAX_SOURCES]; void *src_ud;
    argot_error_fn      on_error;
    argot_help_fn       render_help;
    argot_suggest_fn    suggest;
    argot_unknown_fn    on_unknown_cmd; /* git-style external subcommands */
    const argot_messages *messages;
    void               *arena;  size_t arena_cap;
    unsigned            help_width;     /* 0 = width hook, else 80 */
} argot_config;
```

6.8 Level 2 — pull tokenizer

The tokenizer knows syntax, not the spec. For bundles like `-ofile` it yields `o` and lets the caller decide whether the rest is a value.

```
argot_tokenizer tz;
argot_tok t;
argot_tok_init(&tz, argc, argv, ARGOT_STYLE_GNU);

while (argot_tok_next(&tz, &t)) {
    switch (t.kind) {
    case ARGOT_T_SHORT:      if (t.ch == 'o') out =
argot_tok_take_value(&tz); break;
    case ARGOT_T_LONG:       /* t.name, t.name_len, t.value or NULL */
break;
    case ARGOT_T_POSITIONAL: /* t.value */
break;
    case ARGOT_T_END:        /* saw "--" */
break;
    }
}
```

6.9 Function overview

Function / macro	Level	Purpose
`ARGOT_RUN(argc, argv, name, summary, items...)`	0	Declare, parse, dispatch, return exit code
`ARGOT_RUN_EX(&cfg, argc, argv, ...)`	0	Same, with a config
`argot_init(p, items, cfg)`	1	Prepare a parser; no allocation
`argot_parse(p, argc, argv)`	1	Parse; returns `argot_status`
`argot_report(p)`	1	Print help, version or error; return the exit code
`argot_run(p)`	1	Call the selected command's handler
getters (§6.4)	0, 1	Read values by key
`argot_fail(a, msg)`	0, 1	Report an error from a handler, `ON` or check function
`argot_error(p)`	1	Structured error (§12)
`argot_help(p, path, write, ud)`	1	Render help for any command

Function / macro	Level	Purpose
`argot_completion(p, shell, write, ud)`	1	Emit a completion script
`argot_compile(items, cfg, &err)`	1	Check a spec ahead of time (tests, CI)
`argot_free(p)`	1	Release memory from the alloc hook; no-op otherwise
`argot_tok_init / _next / _take_value`	2	Syntax-only iteration
`argot_type_parse(type, text, dst)`	util	Use built-in converters outside the parser

7. Parsing semantics

argot follows GNU `getopt_long` behaviour by default, with three deliberate differences: `argv` is never permuted, values are typed, and every ambiguity is an error instead of a guess.

7.1 Token grammar

```
arg       = "--"                       ; end of options
          | "-"                        ; positional (stdin)
          | "--" name [ "=" value ]    ; long option
          | "-" short { short } [ value ]  ; bundle; value only after a
value-taking short
          | "@" path                   ; response file, only with
ARGOT_RESPONSE_FILES
          | other                      ; positional or subcommand
name      = 1*( ALNUM | "-" | "_" | "." )   ; ASCII, case-sensitive
short     = any printable ASCII except "-"
```

7.2 Rules

1. `--` ends option parsing; everything after it is positional.
2. A lone `-` is a positional.
3. A value-taking option consumes the next argument even if it starts with `-` (`-j -4`). `ARGOT_STRICT_VALUES` rejects values that match a known option instead.

4. `-<digit>…` is a positional when the command defines no digit short options; this keeps negative numbers usable.
5. In a bundle, the first value-taking short consumes the rest: `-vofile` → `-v`, `-o file`.
6. `--name=` gives an empty string. Valid for strings, an error for numbers.
7. Optional values (`.optional_value`) bind only in attached form: `--color=always`, `-calways`. `--color always` leaves `always` as a positional.
8. A flag given a value (`--verbose=yes`) is an error unless the flag is a bool, which accepts `true/false/yes/no/on/off/1/0`.
9. `--no-name` sets a `.negatable` bool to false. An explicitly declared `--no-name` option wins over the generated one.
10. Repeats: `VALUE` → last wins (or error with `.once`), `LIST` → append, `COUNT` → increment.
11. Ordering: GNU mode interleaves options and positionals. POSIX mode (`ARGOT_STYLE_POSIX`, or `POSIXLY_CORRECT` when `ARGOT_HONOR_POSIXLY_CORRECT` is set) stops option parsing at the first positional.
12. Subcommand boundary: options after a subcommand name belong to that subcommand; parent options `with .global = true` are accepted at any depth.
13. Abbreviation (opt-in): an exact match wins; a unique prefix matches; several matches → `ARGOT_E_AMBIGUOUS` listing candidates.
14. Unknown option → error with suggestion, or, with `ARGOT_ALLOW_UNKNOWN`, passed into `argot_rest()` for wrapper tools.
15. Precedence after the pass: CLI > extra sources (in order) > env > default. Defaults are text parsed by the option's type, so they are checked by the same code as user input.
16. Bytes in, bytes out: names are ASCII, values are arbitrary bytes. UTF-8 matters only for help-text width.

7.3 Edge cases

Input	Spec	Result
`-vvv`	`v` COUNT	level = 3
`-j4`	`j` INT	jobs = 4
`-j -4`	`j` INT	jobs = −4
`-xvf a.tar`	`x`, `v` FLAG, `f` STRING	f = `a.tar`

Input	Spec	Result
`--out=`	`out` STRING	out = `""`
`--jobs=`	`jobs` INT	`ARGOT_E_BAD_VALUE`
`-- -v`	any	positional `-v`
`--verb`	`verbose`, abbrev on	verbose
`--ver`	`verbose` + `version`, abbrev on	`ARGOT_E_AMBIGUOUS`
`--color red`	`color` OPTIONAL_VALUE	color = default, `red` positional
`--no-color`	`color` NEGATABLE	color = false
`-5`	no digit shorts, INT positional	positional = −5
`-`	—	positional `-`

7.4 Algorithm

1. Resolve the command path: start at the root; the compiled index of each visited command is built on first use.
2. Pull tokens. For each option token: look up (short table / long index), check kind, fetch the value per rules 3–8, convert through the type into `dst`, record seen + source, run `ARGOT_ON callbacks`.
3. For each positional token: if it names a subcommand of the current command and no positional of that command is still open, descend; otherwise assign it to the next positional slot.
4. After the last token, for each option not seen: try extra sources, then env, then `def`.
5. Check positional counts, `.required, per-item check`, then command rules (skipped like for --help when an ARGOT_DO flag was given).
6. Return `ARGOT_OK`, `ARGOT_EXIT_HELP`, `ARGOT_EXIT_VERSION` or the first error.

8. Type system and extensibility

A value type is a small `const` vtable; built-in types use exactly the same interface as yours, and decorator types wrap any type to add ranges or checks without new parsing code.

8.1 Type interface

```
typedef struct argot_type {
    const char *name;     /* metavar default, e.g. "N", "FILE", "COLOR"
*/
    size_t      size;     /* bytes per element, used by lists
*/
    int  (*parse)   (const struct argot_type *t, const char *s, size_t n,
                     void *dst, argot_ctx *ctx);          /* 0 = ok, else
argot_fail() */
    int  (*format)  (const struct argot_type *t, const void *src,
                     char *buf, size_t cap);              /* optional: show
defaults   */
    int  (*complete)(const struct argot_type *t, const char *prefix,
                     argot_emit_fn emit, void *ud);       /* optional: shell
completion */
    const void *ud;       /* type data: enum table, range, inner type
*/
    uint32_t    hints;    /* ARGOT_HINT_FILE | ARGOT_HINT_DIR |
ARGOT_HINT_HOST ... */
} argot_type;
```

8.2 Built-in types

Type	C storage	Accepts	Notes
`argot_t_bool`	`bool`	`true false yes no on off 1 0`, case-insensitive	Used for `--flag=value`
`argot_t_i8` … `argot_t_i64`, `argot_t_int`, `argot_t_long`	matching signed type	decimal, `0x`, `0o`, `0b`; optional `_` digit separators	Own locale-free parser; overflow is an error
`argot_t_u8` … `argot_t_u64`, `argot_t_size_t`	matching unsigned type	as above	Rejects a leading `-` (unlike `strtoul`)
`argot_t_double`, `argot_t_float`	`double` / `float`	decimal and exponent forms, `inf`, `nan`	Locale-independent (see §16); stripped by `ARGOT_NO_FLOAT`

Type	C storage	Accepts	Notes
`argot_t_string`	`const char *`	anything	Borrowed from `argv`, no copy
`argot_t_enum`	`int`	one name from a table	Table also feeds help and completion
`argot_t_bytesize`	`uint64_t`	`512`, `4k`, `16M`, `2G`, `1KiB`	Decimal and binary suffixes, overflow-checked
`argot_t_duration`	`uint64_t` (ms)	`250ms`, `2s`, `1m30s`, `1h`	Overflow-checked

8.3 Decorator types

Decorators are types whose `parse` calls an inner type, then checks. They compose:

```
ARGOT_RANGE_INT(1, 256)              /* int within [1, 256]          */
ARGOT_RANGE_DOUBLE(0.0, 1.0)
ARGOT_NONEMPTY(&argot_t_string)      /* rejects ""                    */
ARGOT_MAXLEN(&argot_t_string, 255)
ARGOT_ONE_OF_STR("auto", "always", "never")
```

For the common case nobody writes decorators by hand: `.min` / `.max` on an item (§6.3) wrap its type in the matching range decorator automatically.

8.4 A custom type in 12 lines

```
static int parse_rgb(const argot_type *t, const char *s, size_t n,
                     void *dst, argot_ctx *ctx)
{
    uint32_t v = 0;
    if (n != 7 || s[0] != '#') return argot_fail(ctx, "expected #RRGGBB");
    for (size_t i = 1; i < 7; i++) {
        int d = argot_hexval(s[i]);
        if (d < 0) return argot_fail(ctx, "invalid hex digit");
        v = (v << 4) | (uint32_t)d;
    }
    *(uint32_t *)dst = v;
    return 0;
}
```

```
static const argot_type t_rgb = { .name = "COLOR", .size = sizeof(uint32_t),
.parse = parse_rgb };

/* use it */
ARGOT_CUSTOM('c', "color", &t_rgb, "Accent colour", .def = "#ff8800")

/* read it in a handler */
uint32_t accent;
argot_get(a, "color", &accent);
```

8.5 Hook catalogue

Hook	Default	Replace it to …
`write_out`, `write_err`	`fwrite` to stdout/stderr	log, test, print to a GUI or a UART
`getenv`	`getenv` (or `secure_getenv` where available)	sandbox, test, read wide env on Windows
`alloc`	none — arena only	allow growable lists and large response files
`sources[]`	none	read config files, registry, remote config as extra value layers
`on_error`	format + print to `write_err`	JSON errors, custom wording, telemetry
`render_help`	built-in renderer	Markdown, man page, coloured or boxed help
`messages`	English table	translate or change tone of every message
`suggest`	bounded Damerau–Levenshtein	custom spelling rules, synonyms
`term_width`	`COLUMNS` env, else 80	`ioctl(TIOCGWINSZ)`, Windows console API
`ARGOT_DO / ARGOT_ON`	none	immediate effects: `--load plugin`, `--list-formats`

Hook	Default	Replace it to …
`.check`, rule callbacks	none	cross-field and semantic validation
`ARGOT_CMD handler`	none	dispatch to subcommand handlers with `argot_run()`
`ARGOT_ASSERT(x)`	`assert` in debug, nothing in release	integrate with your own assert system

Introspection rounds this off: `argot_foreach_item()` walks a spec tree, so tools can generate docs, config templates or GUIs from the same declaration.

9. Subcommands

A subcommand is an `ARGOT_CMD` item, and everything that belongs to it — options, arguments, rules, aliases, examples, deeper subcommands — is written inside it. The source reads like the command tree it describes, top to bottom.

```
ARGOT_CMD("repo", NULL, "Manage repositories",          /* NULL: groups only,
a subcommand is required */
    ARGOT_FLAG('q', "quiet", "Less output"),              /* belongs to
"repo" and below */
    ARGOT_CMD("add", repo_add, "Add a repository",
        ARGOT_ARG("name", "Repository name"),
        ARGOT_ARG("url",  "Repository URL"),
        ARGOT_FLAG(0, "trusted", "Skip signature checks"),
        ARGOT_EXAMPLE("pkg repo add main https://example.org", "Add the main
repo")),
    ARGOT_CMD("list", repo_list, "List repositories",
        ARGOT_ALIAS("ls")))
```

9.1 Behaviour

- **Matching:** by name or `ARGOT_ALIAS`; unique prefix with `ARGOT_ABBREV`; unknown names get a “did you mean”.
- **Scope:** an option belongs to the command it is written in. `.global = true` makes it valid at every depth and visible to every handler; it may appear before or after the subcommand name (`pkg -v repo add …` and `pkg repo add … -v`).

- **Handlers:** the deepest selected command's function runs with `argot *a`; getters see its own items and, through the lookup chain (§5.3), all `.global` items above it.
- **Grouping commands:** `NULL` as function means the command only groups others; calling it without a subcommand prints its help and exits with 2. Give it a function instead to make `pkg repo` alone do something (e.g. list).
- **Help:** `pkg repo --help` and `pkg help repo` (with `ARGOT_HELP_COMMAND`) show that level only.
- **Errors** carry the path: `pkg repo add: missing required argument <url>`.
- **External subcommands:** `cfg.on_unknown_cmd` can claim an unknown name and receive the remaining arguments, for git-style `pkg-audit` plugins. argot never executes anything itself.
- **Laziness:** only commands on the selected path get indexes built, so a tree with 200 commands costs nothing until used.

9.2 Dispatch without handlers

Handlers are optional at Level 1: call `argot_parse()` (§6.7), skip `argot_run()` and branch on the selected path. Leaf commands then pass `ARGOT_NO_HANDLER` instead of a function, which tells the spec check that the missing handler is intended.

```
if (argot_parse(&p, argc, argv) != ARGOT_OK)
    return argot_report(&p);

if      (argot_path_is(&p, "install"))  do_install(&p);
else if (argot_path_is(&p, "repo add")) do_repo_add(&p);
```

9.3 Large trees

Each subtree can live in its own source file with `ARGOT_DEFINE` / `ARGOT_EXTERN` / `ARGOT_USE` (§6.6), so a tool with dozens of commands keeps one short `main` and one file per command family.

10. Validation and constraints

Validation runs at three levels — value, option, command — and always after env and defaults are applied, so a rule sees the final picture.

Level	Mechanism	Example
Value	Type parse, `.min` / `.max`, decorator types	`.min = "1", .max = "256"`
Item	Fields + check function	`.required = true`, `.once = true`, `.check = path_exists`
Command	Rule items inside `ARGOT_CMD`	`ARGOT_MUTEX("json", "yaml")`

10.1 Command rules

Rule	Macro	Meaning
Mutually exclusive	`ARGOT_MUTEX("json", "yaml", "toml")`	at most one of them
Exactly one	`ARGOT_ONE_OF("file", "stdin")`	exactly one of them
At least one	`ARGOT_ANY_OF("add", "remove")`	one or more of them
Requires	`ARGOT_REQUIRES("user", "password")`	if `user` then `password`
Conflicts	`ARGOT_CONFLICTS("quiet", "verbose")`	never both
Custom	`ARGOT_RULE_FN(check_fn, ud)`	any logic; message via `argot_fail()`

Rules name options by long name, or `"-x"` for short-only options. Names are resolved to indexes when the spec is compiled; an unknown name is a spec error, not a runtime surprise.

Rules are ordinary items, written inside the command they constrain:

```
ARGOT_CMD("export", do_export, "Export data",
    ARGOT_FLAG(0, "json", "JSON output"),
    ARGOT_FLAG(0, "yaml", "YAML output"),
    ARGOT_STR (0, "user", "User name"),
    ARGOT_STR (0, "password", "Password"),
    ARGOT_MUTEX("json", "yaml"),
    ARGOT_REQUIRES("user", "password"))
```

10.2 What counts as "set"

By default, `.required` and rules count CLI, extra sources and env, but not defaults. `.rule_sources` on the config changes this mask per application.

10.3 Stop or return on error

The default stops at the first error, prints it and exits with its code. Errors are never collected.

This is a compile-time switch, not a runtime option:

```
/* default: print the error, exit with its code */
#include "argot.h"

/* own control: get the status back instead */
#define ARGOT_EXIT_ON_ERROR 0
#include "argot.h"
```

With `ARGOT_EXIT_ON_ERROR 0`, `ARGOT_RUN` and `argot_parse()` return the status and keep the `argot_error` (§12) for `argot_report()`; parsing still stops at the first error. The exit itself goes through `ARGOT_EXIT(code)`, which defaults to `exit(code)` and can be redefined as well, e.g. to flush logs first or to `longjmp` out in tests. The value must be the same in every translation unit, so it is best set in the build system (`-DARGOT_EXIT_ON_ERROR=0`).

11. Help, usage, version and completion

All user-facing text is generated from the spec, streamed through the write hook in small chunks, and never needs a heap buffer.

11.1 Generated help

```
Usage: mytool [OPTIONS] <INPUT>...

Compiles things quickly.

Arguments:
  <INPUT>...              Input files

Options:
  -v, --verbose           More output (repeatable)
  -o, --output <FILE>     Output file [default: out.bin]
  -j, --jobs <N>          Parallel jobs (1-256) [default: 4] [env:
MYTOOL_JOBS]
      --format <FORMAT>   Output format [default: bin]
                          [possible: bin, hex, elf]
  -h, --help              Print help
  -V, --version           Print version

Commands:
  build    Build the project
  clean    Remove build output

See 'mytool <command> --help' for more.
```

- **Usage line:** generated; `[OPTIONS]` collapses long option lists; fully replaceable through `render_help`.
- **Wrapping:** at terminal width (hook) or 80, aligned description column, UTF-8 aware width; East Asian wide characters via an optional `wcwidth` hook.
- **Groups:** `ARGOT_GROUP("Network options")` starts a help section; the items after it join that section.
- **Hidden:** items with `.hidden = true` are parsed but not listed.
- **Annotations:** default (via `type->format`), env name, range, enum choices — each switchable.
- **Examples:** `ARGOT_EXAMPLE` items are listed under an “Examples” heading.
- **Colour:** off by default; `ARGOT_COLOR_AUTO` uses an `isatty` hook and respects `NO_COLOR`.
- **Custom renderer:** `render_help` receives a read-only help model (sections, rows, wrapped cells), so a Markdown or roff renderer reuses the layout logic.

11.2 Version

`-V/--version` prints `cfg.version_text` and returns `ARGOT_EXIT_VERSION`. Disabled automatically when no text is set.

11.3 Shell completion

- **Static scripts:** `argot_completion(p, ARGOT_SH_BASH | _ZSH | _FISH, write, ud)` emits a script from the spec — options, subcommands, enum choices, file/dir hints.
- **Dynamic mode:** a hidden `--argot-complete=<index> -- <words>` entry point returns candidates at runtime, including results of a type's `complete` callback (branch names, device lists).
- **Docs output:** an optional `argot_doc` module renders man pages (roff) and Markdown from the same help model.

12. Error handling

Every failure is a returned `argot_status` plus a structured `argot_error`; by default argot prints the error and exits with its code (ARGOT_EXIT_ON_ERROR, §10.3), and user mistakes are kept strictly apart from programmer mistakes.

12.1 Status codes

Status	Meaning	Exit code
`ARGOT_OK`	Parsed	—
`ARGOT_EXIT_HELP`, `ARGOT_EXIT_VERSION`	Help/version requested	0
`ARGOT_E_UNKNOWN_OPTION`	No such option	2
`ARGOT_E_AMBIGUOUS`	Prefix matches several options	2
`ARGOT_E_MISSING_VALUE`	Option needs a value	2
`ARGOT_E_UNEXPECTED_VALUE`	Flag got `=value`	2
`ARGOT_E_BAD_VALUE`	Type conversion failed	2
`ARGOT_E_RANGE`	Value outside allowed range	2

Status	Meaning	Exit code
`ARGOT_E_REPEATED`	`ARGOT_ONCE` option given twice	2
`ARGOT_E_MISSING_REQUIRED`	Required option or positional absent	2
`ARGOT_E_TOO_MANY_ARGS` / `_TOO_FEW_ARGS`	Positional count	2
`ARGOT_E_UNKNOWN_COMMAND`	No such subcommand	2
`ARGOT_E_RULE`	A command rule failed	2
`ARGOT_E_NO_MEMORY`	Arena/alloc exhausted	70
`ARGOT_E_IO`	Response file unreadable	66
`ARGOT_E_SPEC`	Programmer error in the spec	70
ARGOT_E_ACTION	An ARGOT_ON function returned non-zero	1

Exit codes come from `argot_exit_code()` and are configurable; 2 follows common getopt-based tools, 66/70 follow `sysexits.h`.

12.2 Error object

```
typedef struct argot_error {
    argot_status      status;
    int               argi;        /* index in argv, -1 if not tied to one
*/
    size_t            offset;      /* byte offset inside argv[argi]
*/
    const argot_item *cmd;         /* command being parsed
*/
    const argot_item *item;        /* option involved, may be NULL
*/
    const char       *text;        /* offending input (borrowed)
*/
    size_t            text_len;
    const char       *suggestion;  /* "--jobs", may be NULL
*/
    const char       *detail;      /* from argot_fail(), may be NULL
*/
} argot_error;
```

The message is formatted lazily, only when printed, from the `messages` table:

```
mytool: invalid value 'abc' for '--jobs <N>': expected an integer
mytool: unknown option '--job'
  Did you mean '--jobs'?
Try 'mytool --help' for more information.
```

12.3 Programmer errors

Spec mistakes — duplicate keys, an option with neither short nor long name, `.def` / `.min` / `.max` text the type cannot parse, a `.to` variable of the wrong type, rules naming unknown keys, a leaf command without handler — are detected by the spec compiler. Getter misuse (unknown key, wrong getter for the type) is caught on the first call. Debug builds trip `ARGOT_ASSERT` with the exact item; release builds return `ARGOT_E_SPEC`. `argot_compile()` in a unit test catches spec mistakes before shipping.

13. Memory model

The default path allocates nothing: strings are borrowed from `argv`, working state lives in an arena inside the parser struct, and anything bigger needs an explicit caller decision.

13.1 Three tiers

1. **Inline arena** — `ARGOT_INLINE_ARENA` bytes (default 1024) inside `argot_parser`. Enough for typical tools.
2. **Caller arena** — `cfg.arena` / `cfg.arena_cap`: any buffer, e.g. a static array on a microcontroller.
3. **Alloc hook** — `cfg.alloc(ud, ptr, old, new)` (realloc-style, free when `new == 0`). Only used when the arena is exhausted or a list is growable. Released by `argot_free()`.

When all three are exhausted the parse fails with `ARGOT_E_NO_MEMORY`. Nothing is ever silently truncated.

13.2 What lives where

Data	Location	Size
Values bound with .to	caller `dst`	type size
String values	borrowed from `argv`	0
Per-option state (seen, source, count)	arena	4 bytes per option on the parsed path
Short-option index	arena	128 bytes per visited command
Long-option index	arena	2 bytes per long name per visited command
Fixed lists	caller array	`cap × type size`
Growable lists	alloc hook	grows ×2
Split values (`--tags=a,b`)	arena / alloc	value length + 1 (NUL-terminated copies)
Response file contents	arena / alloc	file size + 1
Values of items without .to	arena	type size per item on the parsed path

13.3 Lifetimes

- Values pointing into `argv` live as long as `argv` (the program).
- Values in the arena live as long as the parser object.
- `argot_free()` is idempotent and only needed with an alloc hook; calling it always is harmless.

14. Performance design

Parsing is one linear pass with constant-time short lookups and logarithmic long lookups; everything optional (help, completion, floats, suggestions) costs nothing unless it is used.

Technique	Effect
Specs are `static const`	Zero init cost; live in `.rodata`/flash
Lazy compilation per visited command	Large command trees cost only what the user touches
Short index: `uint8_t[128]` → option slot	O(1) lookup, 128 bytes
Long index: sorted by (length, bytes), binary search	O(log m); linear scan below 8 options, where it is faster
Cached name lengths in the index	No `strlen` on spec names during parsing
One pass over `argv`, bitset post-pass	Defaults/env touched only for unset options
Own integer parser	No locale lookups, no `errno` juggling, branch-light
Lazy error formatting	Errors cost a struct write until printed
Help streamed in 256-byte chunks	No buffer proportional to help size
Suggestion engine only on failure, bounded	Never on the success path
Compile-time strip switches	`ARGOT_NO_HELP`, `ARGOT_NO_COMPLETION`, `ARGOT_NO_FLOAT`, `ARGOT_NO_SUGGEST`
Getter lookups by key	Binary search in the already-built index, length compared first; a handful of comparisons per call

14.1 Benchmarks

A `bench/` suite measures parse time and code size for argv lengths of 10, 1 000 and 100 000 arguments against `getopt_long` and other C parsers. Target: within 1.5× of `getopt_long` on the plain-flag case while also doing typed conversion. Results are published per release; regressions over 10 % fail CI.

15. Security design

argot treats `argv`, the environment and response files as attacker-controlled input — setuid helpers, CGI wrappers and scripts forwarding user strings are all real deployments.

Risk	Mitigation
Integer overflow in conversion	Checked arithmetic in own parser; explicit ranges; `-1` rejected for unsigned types (a classic `strtoul` pitfall)
Buffer overflow in lists or arena	Every write bounds-checked against `cap`; failure is an error, never truncation
Format-string injection	User text is never used as a format string; the internal formatter only substitutes `%s`-style slots from the message table
Terminal escape injection	Non-printable bytes in echoed input are shown as `\xNN` in error messages
Option injection (`-rf` as a filename)	`--` always supported; `ARGOT_STRICT_VALUES`; docs show the safe pattern
Malicious response files	Off by default; nesting depth limit (default 8); size limit; cycle detection
Untrusted environment	`getenv` hook defaults to `secure_getenv` where available; `ARGOT_NO_ENV` disables env fallback entirely
Malformed `argv`	`argc < 0`, `argv[i] == NULL` and `argv[0] == NULL` handled; program name falls back to `cmd.name`
Denial of service	All algorithms linear in input; suggestions only compare names ⩽ 64 bytes with early exit

Risk	Mitigation
Data races	No globals; parsers share nothing but `const` specs
Broken specs	Spec compiler rejects duplicate `keys`, `unparsable defaults or bounds`, `conflicting field`s

Hardened CI build: `-D_FORTIFY_SOURCE=3 -fstack-protector-strong -fsanitize=address,undefined`, plus continuous fuzzing (§17).

16. Portability, build configuration and distribution

One set of C99 sources builds everywhere from a desktop to a bare-metal MCU, and ships both as a single header and as a classic library.

16.1 Language and platforms

- **C99 core.** C11 adds `_Generic` (`ARGOT_TO`) and `static_assert`; C23 adds `[[nodiscard]]` on status-returning functions. All detected via `__STDC_VERSION__`.
- **Macros in strict C99.** Leaf macros expand to `.help = __VA_ARGS__`, so optional fields simply continue the initialiser list and no macro ever needs an empty `__VA_ARGS__`. `ARGOT_CMD` and `ARGOT_RUN` split help text and children with a FIRST/REST preprocessor helper and append the `ARGOT_I_END` terminator. Nested children are compound literals: in block scope they live until `main` returns, at file scope they are static.
- **C++:** headers are `extern "C"`-wrapped and compile as C++11+. Nested compound literals are not valid C++, so C++ code uses Level 1 arrays or a future wrapper header (see Open questions).
- **Targets:** Linux, BSDs, macOS, WASI, `arm-none-eabi` and other freestanding toolchains. Windows (MSVC, MinGW, clang-cl) follows in 0.2.0.
- **Windows:** optional `argot_win_argv()` builds a UTF-8 `argv` from `GetCommandLineW()`.
- **Locale:** integer parsing is locale-free by construction; floats use `strtod_l` / `_strtod_l` with the C locale where available, otherwise a grammar pre-check plus decimal-point swap.

16.2 Configuration macros

Macro	Effect
`ARGOT_IMPLEMENTATION`	Emit the implementation (single-header mode) in this translation unit
`ARGOT_STATIC`	Give all functions `static` linkage
`ARGOT_FREESTANDING`	No `stdio.h` / `stdlib.h`; I/O and env only via hooks; implies ARGOT_EXIT_ON_ERROR 0
`ARGOT_NO_HELP`, `ARGOT_NO_COMPLETION`, `ARGOT_NO_SUGGEST`, `ARGOT_NO_FLOAT`, `ARGOT_NO_ENV`	Strip features
`ARGOT_INLINE_ARENA`	Inline arena size in bytes (default 1024)
`ARGOT_MAX_DEPTH`	Maximum subcommand nesting (default 16)
`ARGOT_MAX_SOURCES`	Extra value-source slots (default 4)
`ARGOT_ASSERT(x)`	Custom assertion
ARGOT_EXIT_ON_ERROR	1 (default): print the error and exit; 0: return the status to the caller (§10.3)
ARGOT_EXIT(code)	How the program ends on an error; default exit(code)

16.3 Distribution

- **Single header:** `dist/argot.h`, generated by an amalgamation script (stb style).
- **Library:** CMake and Meson builds plus a plain Makefile; installs `pkg-config` and CMake package files.
- **Packages after 0.1.0:** vcpkg, Conan, xmake.

```
argot/
  include/argot.h        public API
  src/                   tokenizer.c parse.c types.c rules.c help.c
                         complete.c error.c suggest.c win.c
  tools/amalgamate.py    builds dist/argot.h
  tests/                 unit/ golden/ fuzz/ compile-fail/
  bench/
  examples/              00-minimal ... 09-plugins
  docs/
```

17. Testing strategy

Correctness is proven by table-driven unit tests and golden files, robustness by continuous fuzzing, and portability by a wide compiler matrix.

Layer	What	How
Unit	Tokenizer, each type, matcher, rules	Table-driven: `argv` in → tokens/values/status out
Golden	Help, usage, errors, completion scripts	Snapshot files; diff on change, update with a flag
Property	Type round-trips; GNU mode order-independence	`format` → `parse` equals input; shuffled options give same result
Differential	Common subset vs `getopt_long`	Same `argv`, same flags/values/positionals
Compile-fail	Type-checked macros	Wrong handler signatures in ARGOT_CMD / ARGOT_DO / ARGOT_ON must not compile
Fuzz	Tokenizer, full parse with a rich spec, spec compiler with random specs	libFuzzer + AFL++; short run per push, long run nightly
Sanitizers	All unit and fuzz tests	ASan, UBSan, MSan; Valgrind on the freestanding build
Matrix	Compilers and standards	GCC, Clang, TCC, cproc (MSVC from 0.2.0) × `-std=c99/c11/c17/c23` × C++11/17/20

Layer	What	How
Embedded	Freestanding build	`arm-none-eabi` build with size report in CI

Coverage gates: ⩾ 95 % lines, ⩾ 90 % branches (NFR-10).

18. Versioning and ABI stability

argot uses Semantic Versioning and starts at **0.1.0**; Windows support follows in **0.2.0**. During 0.x a minor release may break the API (always listed in the changelog), a patch release never does. Three rules keep breakage rare even now:

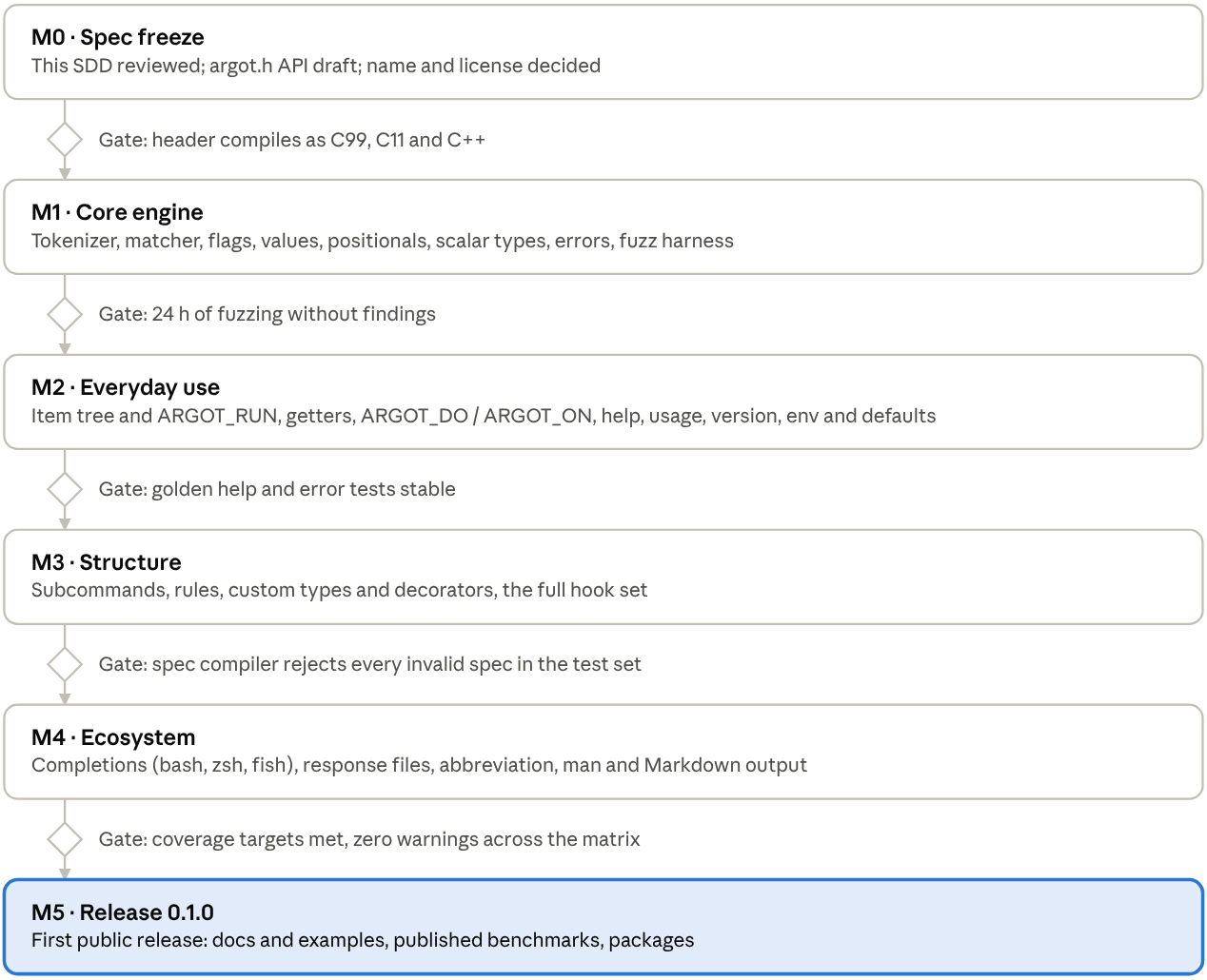
1. **Append-only public structs.** `argot_item` and `argot_type` only gain fields at the end, and zero always means “old behaviour”. Code using the macros or designated initialisers keeps compiling and keeps its meaning.
2. **Versioned config.** `ARGOT_CONFIG_INIT` sets `cfg.version`; the library reads only the fields that version knows, so an old binary with a new library (or the reverse) stays safe.
3. **Opaque parser with reserve.** `argot_parser` exposes its size for stack allocation but reserves padding in `_p`, so private state can grow without changing `sizeof`.

Deprecation: a symbol is marked `ARGOT_DEPRECATED("use X")` for at least one minor release before it is removed. `ARGOT_VERSION_MAJOR/MINOR/PATCH` and `argot_version()` allow compile-time and run-time checks.

19. Roadmap and milestones

Six milestones lead to the first release, 0.1.0; each ends at a gate that must pass before the next one starts. No dates yet — they depend on available time.

Six milestones to 0.1.0, each closed by a gate



argot roadmap · 6 milestones, 5 gates

M1 already gives a usable Level 1 + Level 2 library; everything after it adds convenience and reach, not core semantics. Windows support (MSVC/MinGW builds, UTF-8 argv, console width) is the main goal of 0.2.0.

20. Open questions and comparison

20.1 Open questions

- [ ] C++: nested compound literals are not C++ — offer a C++ wrapper header, or support C++ only through Level 1 arrays?

- [ ] Windows `/option` syntax: decide together with Windows platform support in 0.2.0 — a style flag, or never?
- [ ] Floats: own locale-free parser (more code, exact control) or `strtod_l` with fallback (proposed)?
- [ ] Usage-error exit code: 2 (proposed) or `EX_USAGE` 64?
- [ ] Should `.global` options be allowed before *and* after subcommands by default?
- [ ] Getter keys are strings checked at startup. Also offer a code generator that emits typed accessors (`args.install.force`) for zero-lookup access?

20.2 Comparison with existing C libraries

From the libraries' documentation as known to the author; verify before publishing.

Library	Style	Typed values	Subcommands	Help output	Global state
POSIX `getopt`	Loop	No	No	No	Yes (`optind`)
GNU `getopt_long`	Loop	No	No	No	Yes; permutes `argv`
glibc `argp`	Callbacks	No (strings)	No (child parsers merge options)	Yes	Some; glibc only
cofyc/argparse	Declarative table	int, float, string, bool	Manual	Yes	No
likle/cargs	Table + loop	No	No	Yes	No
skeeto/optparse	Loop	No	No	No	No
argot (this design)	Declarative + loop	Yes, extensible	Yes, nested	Yes + completions	No